

```
In [8]: import pandas as pd
import matplotlib.pyplot as plt

notes = pd.read_parquet('../data/intermediate/notes_enriched.parquet')
ratings = pd.read_parquet('../data/intermediate/ratings_enriched.parquet')
requests = pd.read_parquet('../data/intermediate/requests_enriched.parquet')
# note_status = pd.read_parquet('../data/intermediate/note_status.parquet')
```

```
In [10]: # Considering each post that ever had a note attached
attached = notes.dropna(subset=["noteFirstCrhTime"]).copy()

# find the time that a note was first attached
first_attach = (
    attached.groupby("tweetId")["noteFirstCrhTime"]
    .min()
    .reset_index(name="firstAttachTime")
)

# merge back
df = notes.merge(first_attach, on="tweetId", how="inner")

# notes written BEFORE first attachment
before_df = df[df["noteTime"] < df["firstAttachTime"]].copy()
```

```
In [11]: before_df['classification'].value_counts()
```

```
Out[11]: classification
MISINFORMED_OR_POTENTIALLY_MISLEADING    317826
NOT_MISLEADING                            39316
Name: count, dtype: int64
```

```
In [19]: props = (
    before_df
    .groupby(["tweetId", "classification"])
    .size()
    .unstack(fill_value=0)
)

props["total"] = props.sum(axis=1)

props["misleading_prop"] = (
    props["MISINFORMED_OR_POTENTIALLY_MISLEADING"] / props["total"]
)

props["not_misleading_prop"] = (
    props["NOT_MISLEADING"] / props["total"]
)

props['misleading_prop'].describe()
```

```
Out[19]: count      220834.000000
         mean        0.941137
         std         0.155159
         min         0.000000
         25%         1.000000
         50%         1.000000
         75%         1.000000
         max         1.000000
         Name: misleading_prop, dtype: float64
```

```
In [16]: after_df = df[df["noteTime"] > df["firstAttachTime"]].copy()
         after_df['classification'].value_counts()
```

```
Out[16]: classification
         MISINFORMED_OR_POTENTIALLY_MISLEADING    25739
         NOT_MISLEADING                          23476
         Name: count, dtype: int64
```

```
In [18]: props_a = (
         after_df
         .groupby(["tweetId", "classification"])
         .size()
         .unstack(fill_value=0)
         )

         props_a["total"] = props_a.sum(axis=1)

         props_a["misleading_prop"] = (
             props_a["MISINFORMED_OR_POTENTIALLY_MISLEADING"] / props_a["total"]
         )

         props_a["not_misleading_prop"] = (
             props_a["NOT_MISLEADING"] / props_a["total"]
         )

         props_a['misleading_prop'].describe()
```

```
Out[18]: count      34513.000000
         mean        0.528040
         std         0.467095
         min         0.000000
         25%         0.000000
         50%         0.500000
         75%         1.000000
         max         1.000000
         Name: misleading_prop, dtype: float64
```

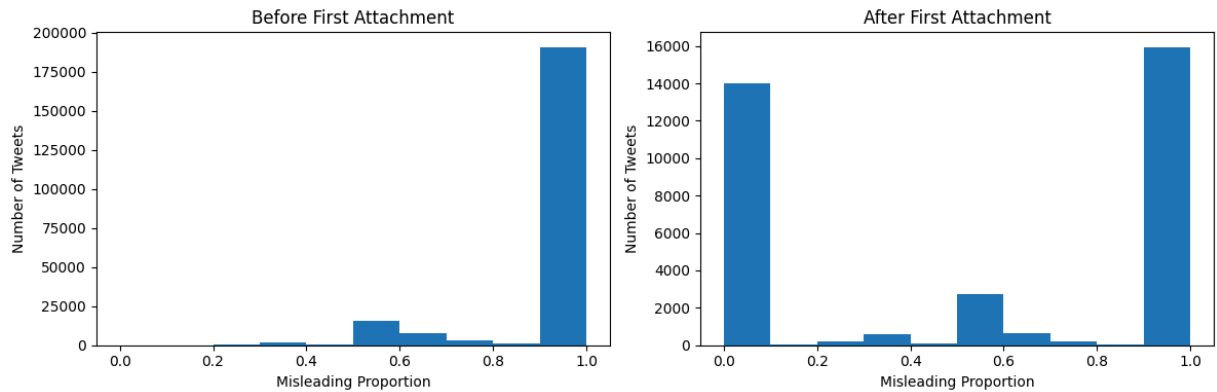
```
In [47]: fig, ax = plt.subplots(1, 2, figsize=(12, 4))

         # Before first attachment
         ax[0].hist(props["misleading_prop"], bins=10)
         ax[0].set_xlabel("Misleading Proportion")
         ax[0].set_ylabel("Number of Tweets")
         ax[0].set_title("Before First Attachment")

         # After first attachment
```

```
ax[1].hist(props_a["misleading_prop"], bins=10)
ax[1].set_xlabel("Misleading Proportion")
ax[1].set_ylabel("Number of Tweets")
ax[1].set_title("After First Attachment")

plt.tight_layout()
plt.show()
```



The decreasing misleading proportion pattern could suggest that once note was attached, meaning contextual information became publicly available, contributors were more likely to converge toward a shared interpretation of the post, resulting in fewer later notes labeling the tweet as misleading. However, this result shows only an association. Misleading classifications still dominate overall despite the downward trend.

```
In [39]: before_ratings = ratings[ratings["ratingTime"] < ratings["noteFirstCrhTime"]]

before_props = (
    before_ratings
    .groupby(["ratedOnTweetId", "helpfulnessLevel"])
    .size()
    .unstack(fill_value=0)
)

before_props["total"] = before_props.sum(axis=1)

before_props["helpful_prop"] = (
    before_props["HELPFUL"] /
    before_props["total"]
)

before_props["not_helpful_prop"] = (
    before_props["NOT_HELPFUL"] /
    before_props["total"]
)

before_props['helpful_prop'].describe()
```

```
Out[39]: count      258731.000000
         mean        0.870945
         std         0.164728
         min         0.000000
         25%         0.833333
         50%         0.910256
         75%         0.967213
         max         1.000000
         Name: helpful_prop, dtype: float64
```

```
In [38]: after_ratings = ratings[ratings["ratingTime"] >= ratings["noteFirstCrhTime"]

after_props = (
    after_ratings
    .groupby(["ratedOnTweetId", "helpfulnessLevel"])
    .size()
    .unstack(fill_value=0)
)

after_props["total"] = after_props.sum(axis=1)

after_props["helpful_prop"] = (
    after_props["HELPFUL"] /
    after_props["total"]
)

after_props["not_helpful_prop"] = (
    after_props["NOT_HELPFUL"] /
    after_props["total"]
)

after_props['helpful_prop'].describe()
```

```
Out[38]: count      259884.000000
         mean        0.789582
         std         0.262060
         min         0.000000
         25%         0.714286
         50%         0.882353
         75%         0.983884
         max         1.000000
         Name: helpful_prop, dtype: float64
```

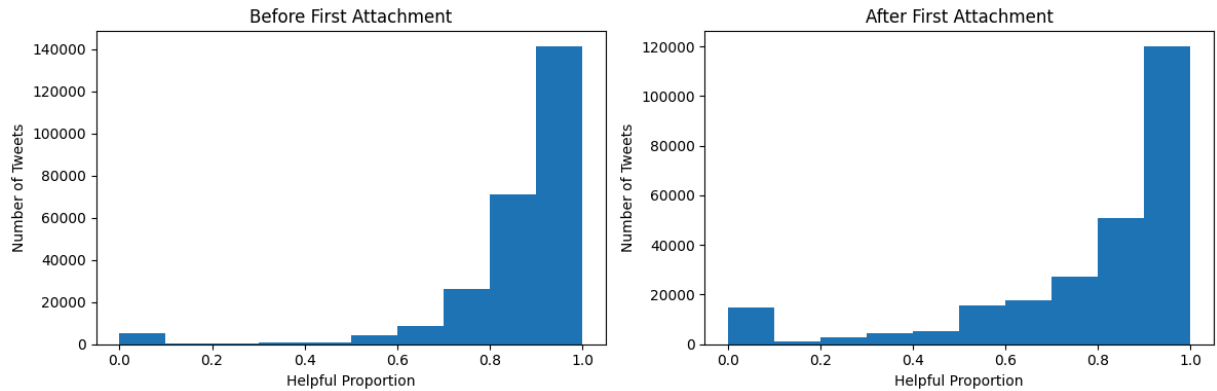
```
In [44]: # HELPFUL PROPORTION
fig, ax = plt.subplots(1, 2, figsize=(12, 4))

# Before first attachment
ax[0].hist(before_props["helpful_prop"], bins=10)
ax[0].set_xlabel("Helpful Proportion")
ax[0].set_ylabel("Number of Tweets")
ax[0].set_title("Before First Attachment")

# After first attachment
ax[1].hist(after_props["helpful_prop"], bins=10)
ax[1].set_xlabel("Helpful Proportion")
ax[1].set_ylabel("Number of Tweets")
```

```
ax[1].set_title("After First Attachment")
```

```
plt.tight_layout()
plt.show()
```



After attachment, the distribution becomes slightly more dispersed, suggesting that once notes become publicly visible, there is somewhat more variation in how later raters evaluate their helpfulness. However, this could be due to sampling sizes since there are typically more ratings after attachment, which naturally introduces more variability.

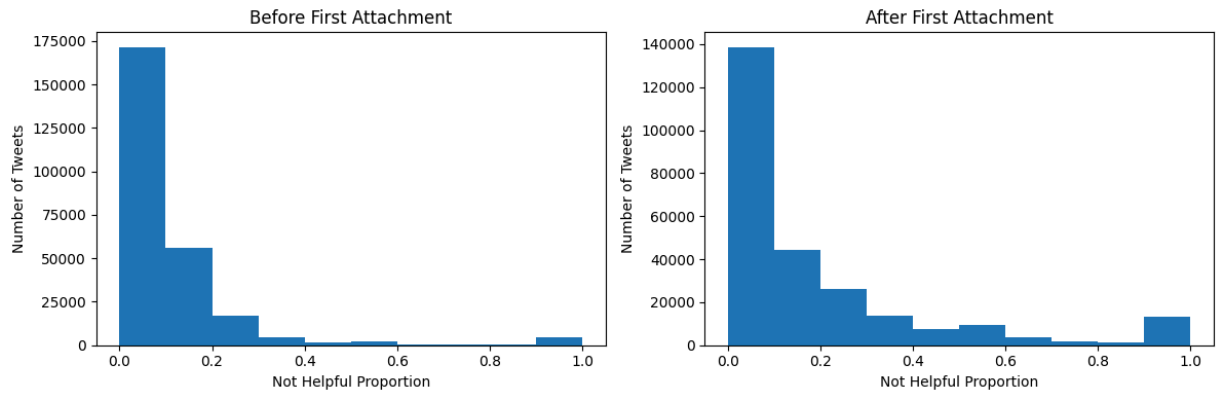
This suggests that posts which receive notes are associated with more heterogeneous evaluations. The notes are affecting how people rate

```
In [ ]: # NOT HELPFUL PROPORTION
fig, ax = plt.subplots(1, 2, figsize=(12, 4))

# Before first attachment
ax[0].hist(before_props["not_helpful_prop"], bins=10)
ax[0].set_xlabel("Not Helpful Proportion")
ax[0].set_ylabel("Number of Tweets")
ax[0].set_title("Before First Attachment")

# After first attachment
ax[1].hist(after_props["not_helpful_prop"], bins=10)
ax[1].set_xlabel("Not Helpful Proportion")
ax[1].set_ylabel("Number of Tweets")
ax[1].set_title("After First Attachment")

plt.tight_layout()
plt.show()
```



```
In [45]: import matplotlib.pyplot as plt

data = [
    before_props["helpful_prop"],
    after_props["helpful_prop"]
]

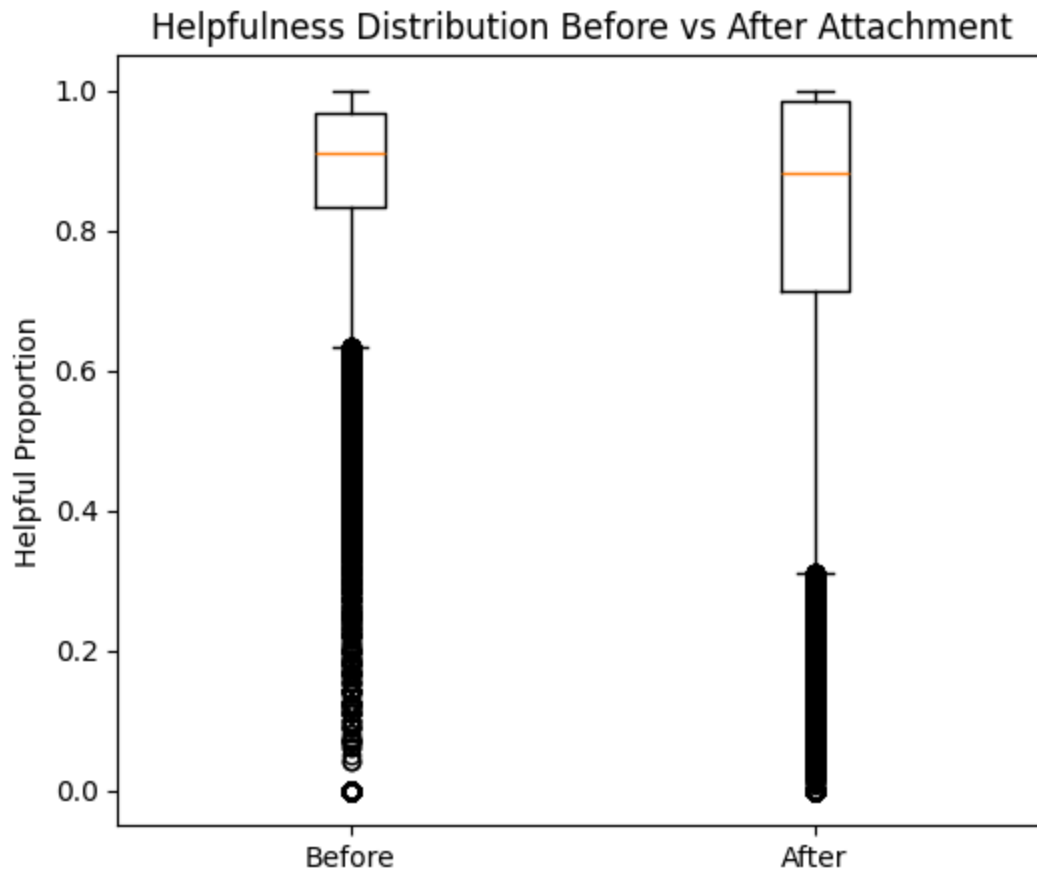
plt.figure(figsize=(6, 5))
plt.boxplot(data, labels=["Before", "After"])

plt.ylabel("Helpful Proportion")
plt.title("Helpfulness Distribution Before vs After Attachment")

plt.show()
```

/tmp/ipykernel_3709093/3611565784.py:9: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.

```
plt.boxplot(data, labels=["Before", "After"])
```



In [46]: `import matplotlib.pyplot as plt`

```
data = [
    before_props["not_helpful_prop"],
    after_props["not_helpful_prop"]
]

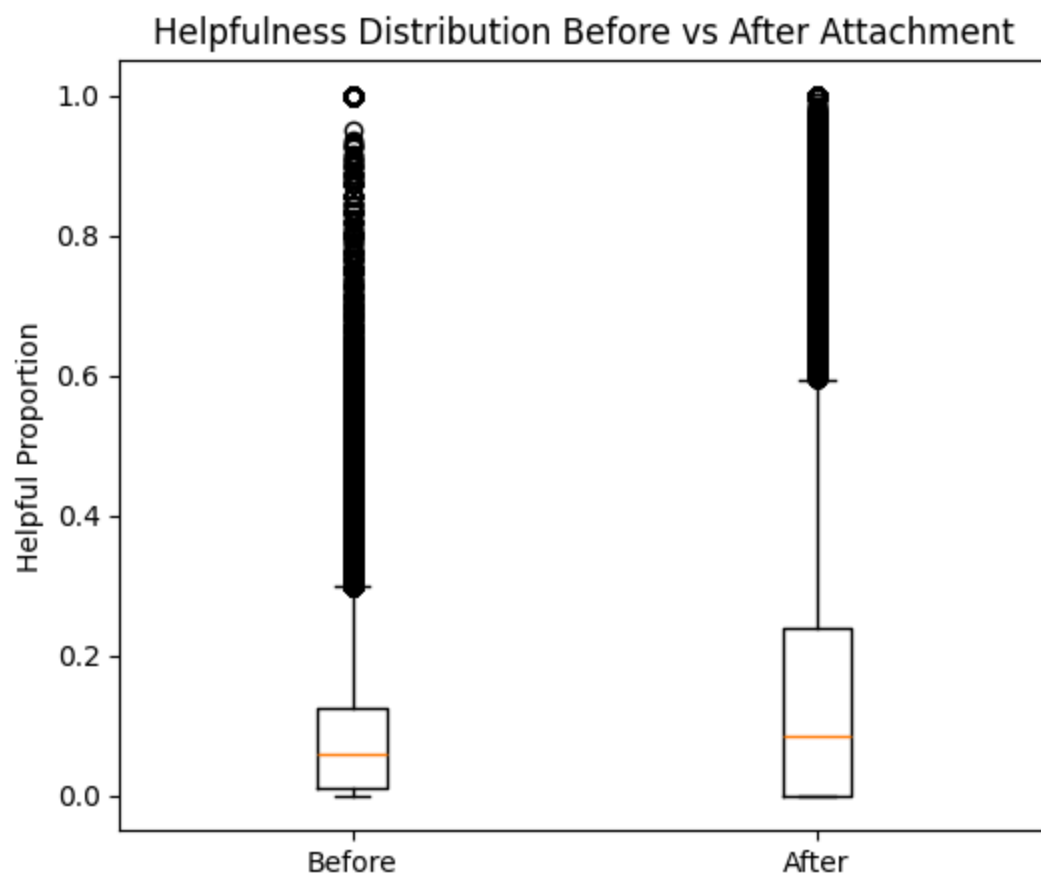
plt.figure(figsize=(6, 5))
plt.boxplot(data, labels=["Before", "After"])

plt.ylabel("Helpful Proportion")
plt.title("Helpfulness Distribution Before vs After Attachment")

plt.show()
```

/tmp/ipykernel_3709093/2145992283.py:9: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.

```
plt.boxplot(data, labels=["Before", "After"])
```



In []: